Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

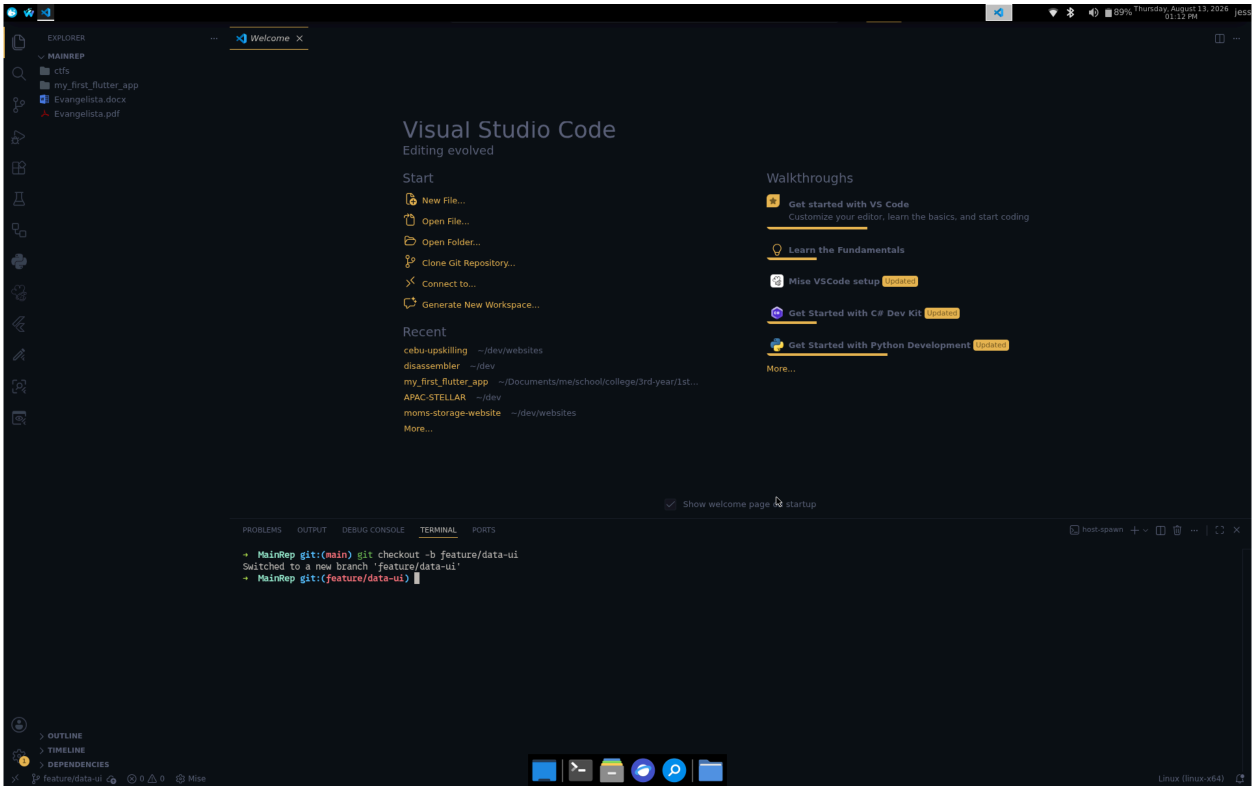
Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch



🏴 Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

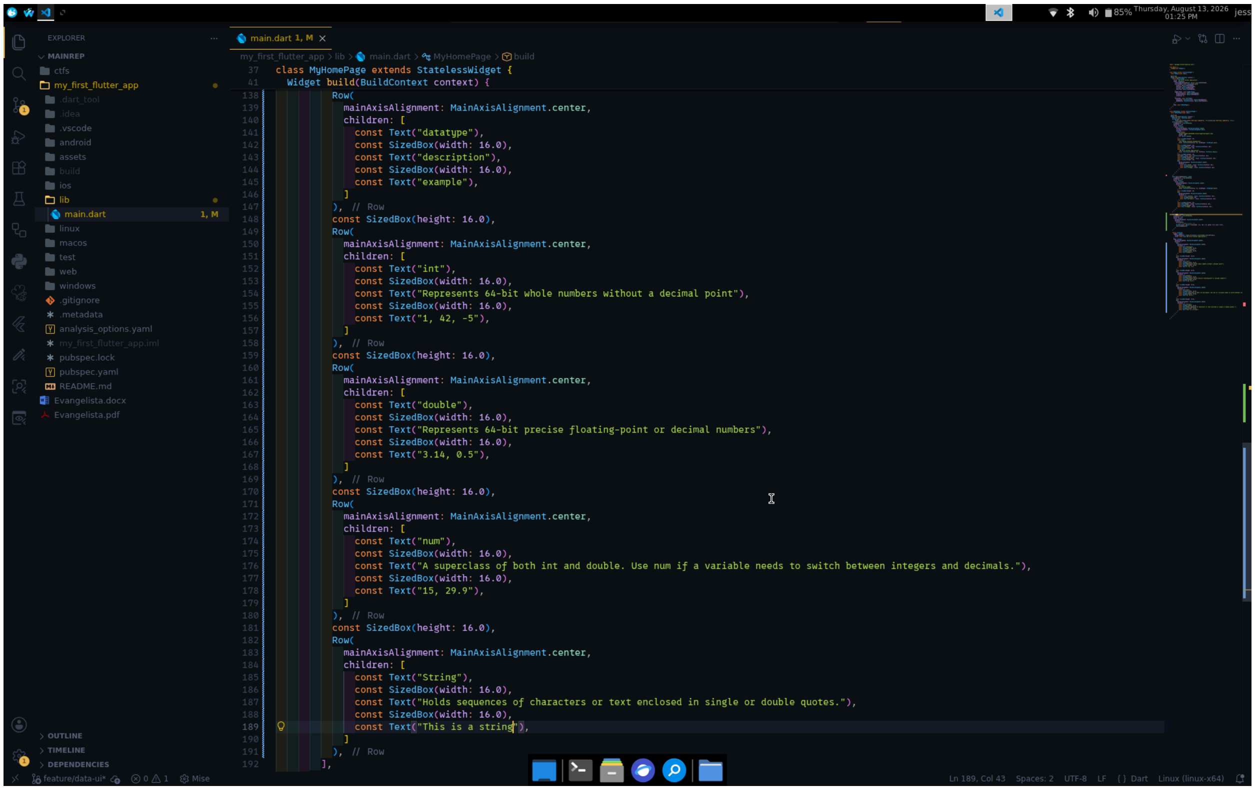
Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Screenshot:

- Your identified data
- The corresponding data types


main.dart 1, M
my_first_flutter_app > lib > main.dart > MyHomePage > build
class MyHomePage extends StatelessWidget {
Widget build(BuildContext context) {
Row(
mainAxisAlignment: MainAxisAlignment.center,
children: [
const Text("datatype"),
const SizedBox(width: 16.0),
const Text("description"),
const SizedBox(width: 16.0),
const Text("example"),
]
), // Row
const SizedBox(height: 16.0),
Row(
mainAxisAlignment: MainAxisAlignment.center,
children: [
const Text("int"),
const SizedBox(width: 16.0),
const Text("Represents 64-bit whole numbers without a decimal point"),
const SizedBox(width: 16.0),
const Text("1, 42, -5"),
]
), // Row
const SizedBox(height: 16.0),
Row(
mainAxisAlignment: MainAxisAlignment.center,
children: [
const Text("double"),
const SizedBox(width: 16.0),
const Text("Represents 64-bit precise floating-point or decimal numbers"),
const SizedBox(width: 16.0),
const Text("3.14, 0.5"),
]
), // Row
const SizedBox(height: 16.0),
Row(
mainAxisAlignment: MainAxisAlignment.center,
children: [
const Text("num"),
const SizedBox(width: 16.0),
const Text("A superclass of both int and double. Use num if a variable needs to switch between integers and decimals."),
const SizedBox(width: 16.0),
const Text("15, 29.9"),
]
), // Row
const SizedBox(height: 16.0),
Row(
mainAxisAlignment: MainAxisAlignment.center,
children: [
const Text("String"),
const SizedBox(width: 16.0),
const Text("Holds sequences of characters or text enclosed in single or double quotes."),
const SizedBox(width: 16.0),
const Text("This is a string"),
]
), // Row
],


my_first_flutter_app
My First Flutter Application
DEBUG
datatype description example
int Represents 64-bit whole numbers without a decimal point 1, 42, -5
double Represents 64-bit precise floating-point or decimal numbers 3.14, 0.5
num A superclass of both int and double. Use num if a variable needs to switch between integers and decimals. 15, 29.9
String Holds sequences of characters or text enclosed in single or double quotes. This is a string

🏴 Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

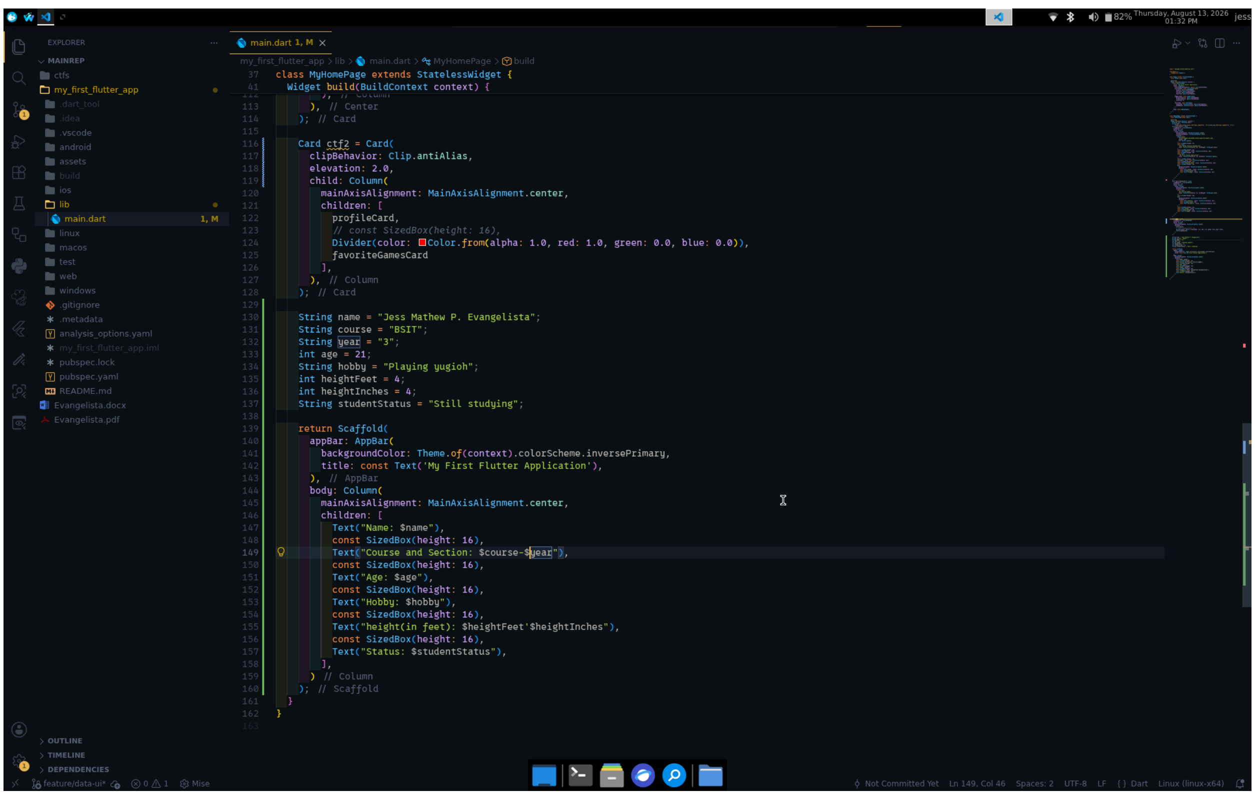
Create variables for at least:

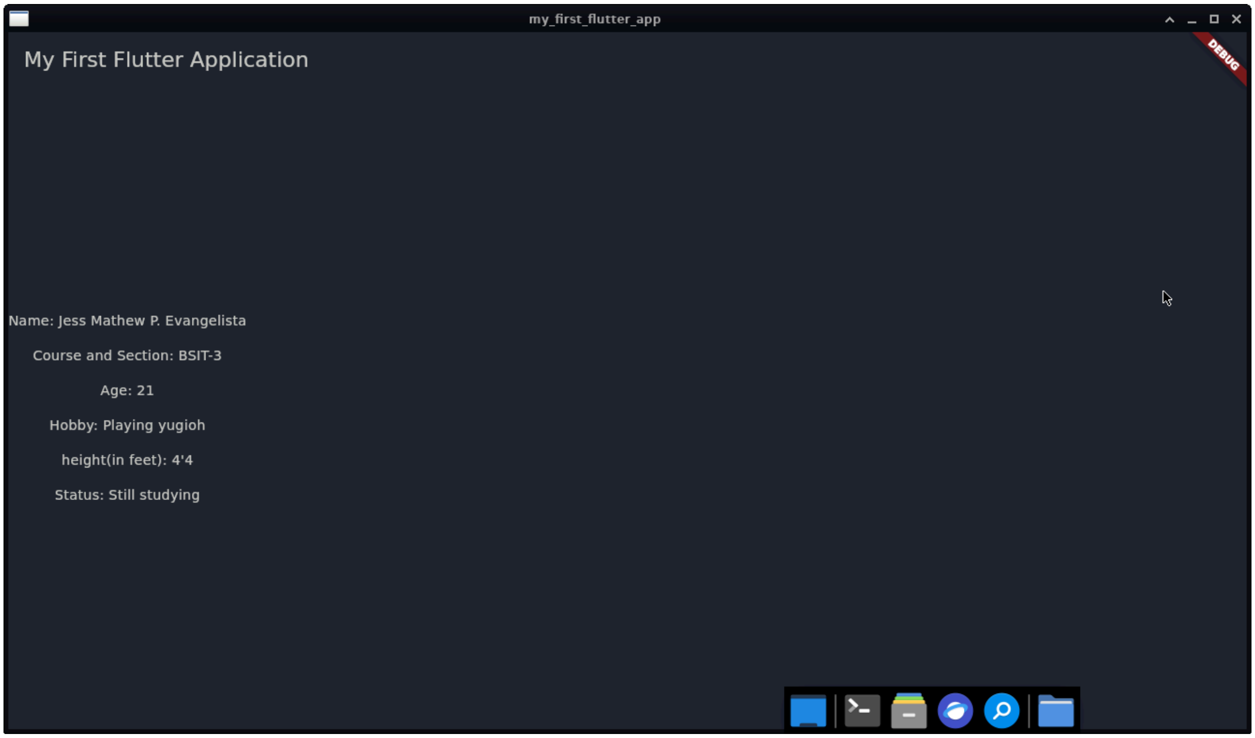
- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

Screenshot:

- Your variables
- Their data types
- Application still running





🏴 Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

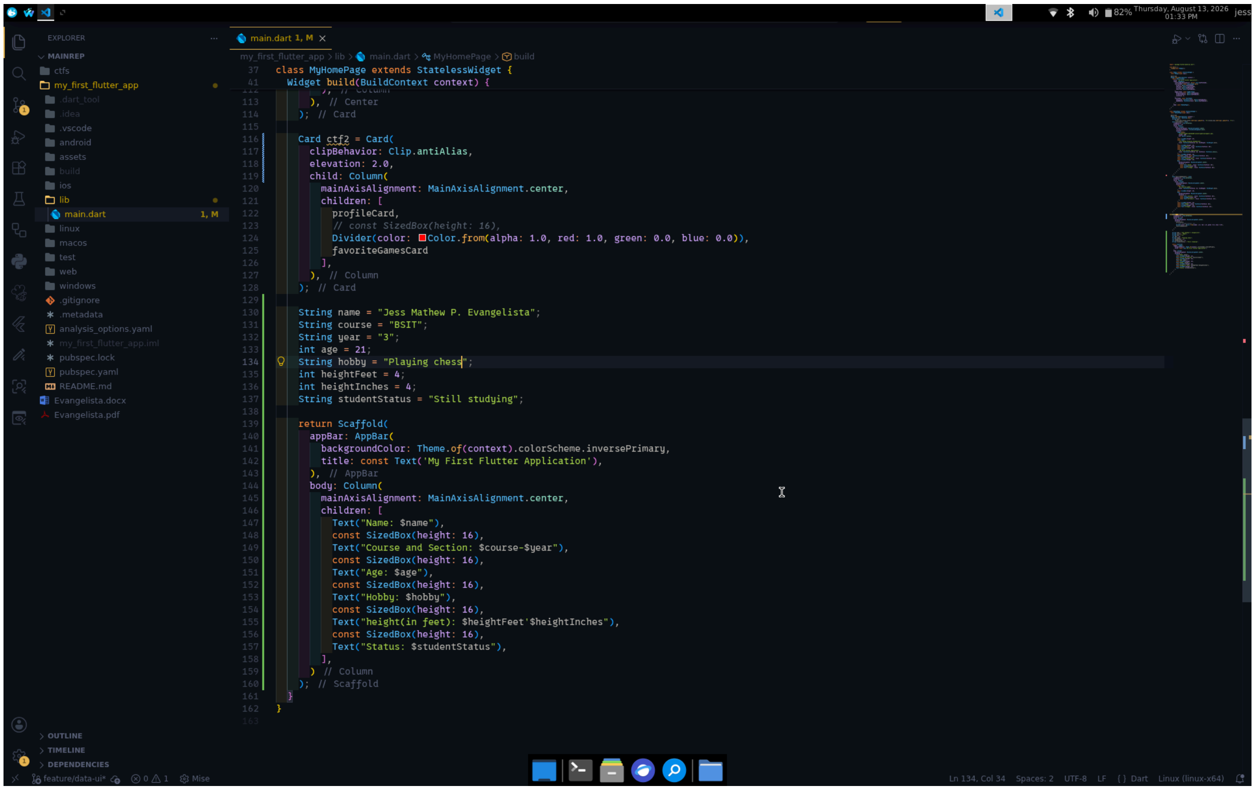
Test:

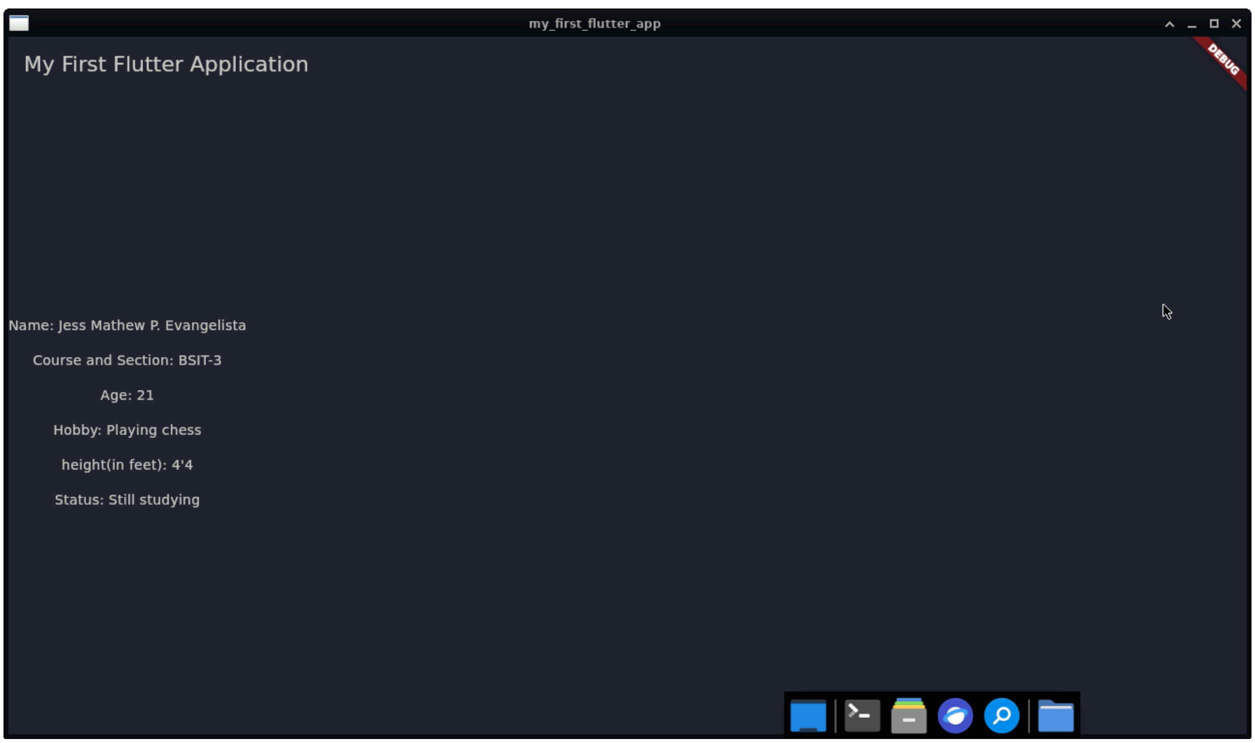
Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables
- Updated application





🏴 Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

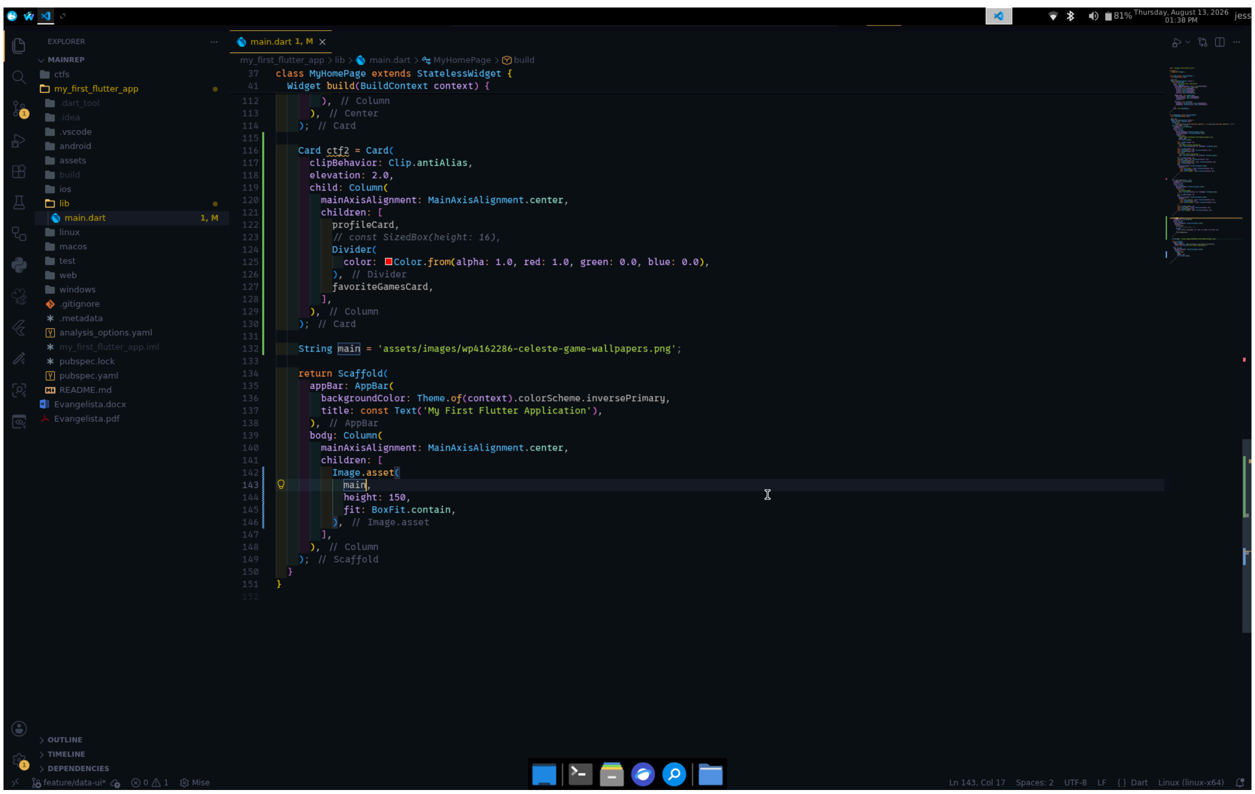
Use the variable when displaying t he image.

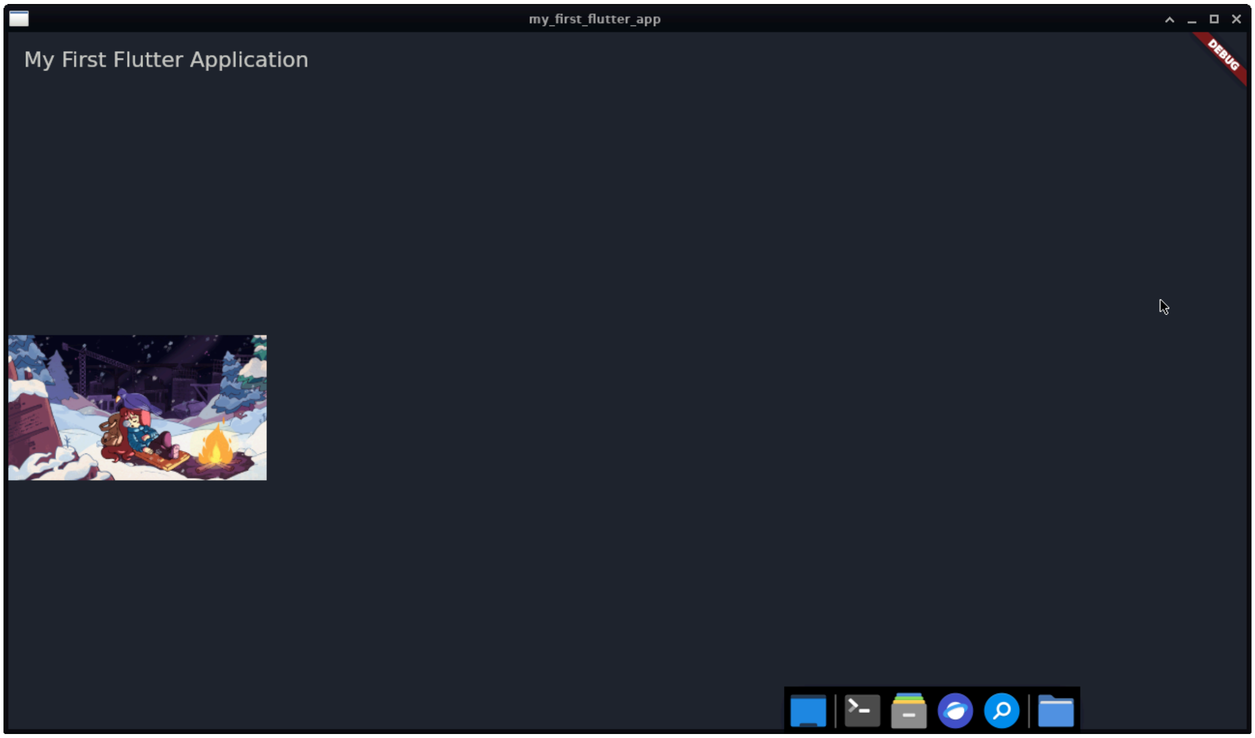
Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

- Image variable
- Image widget using the variable
- Updated application





🏴 Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

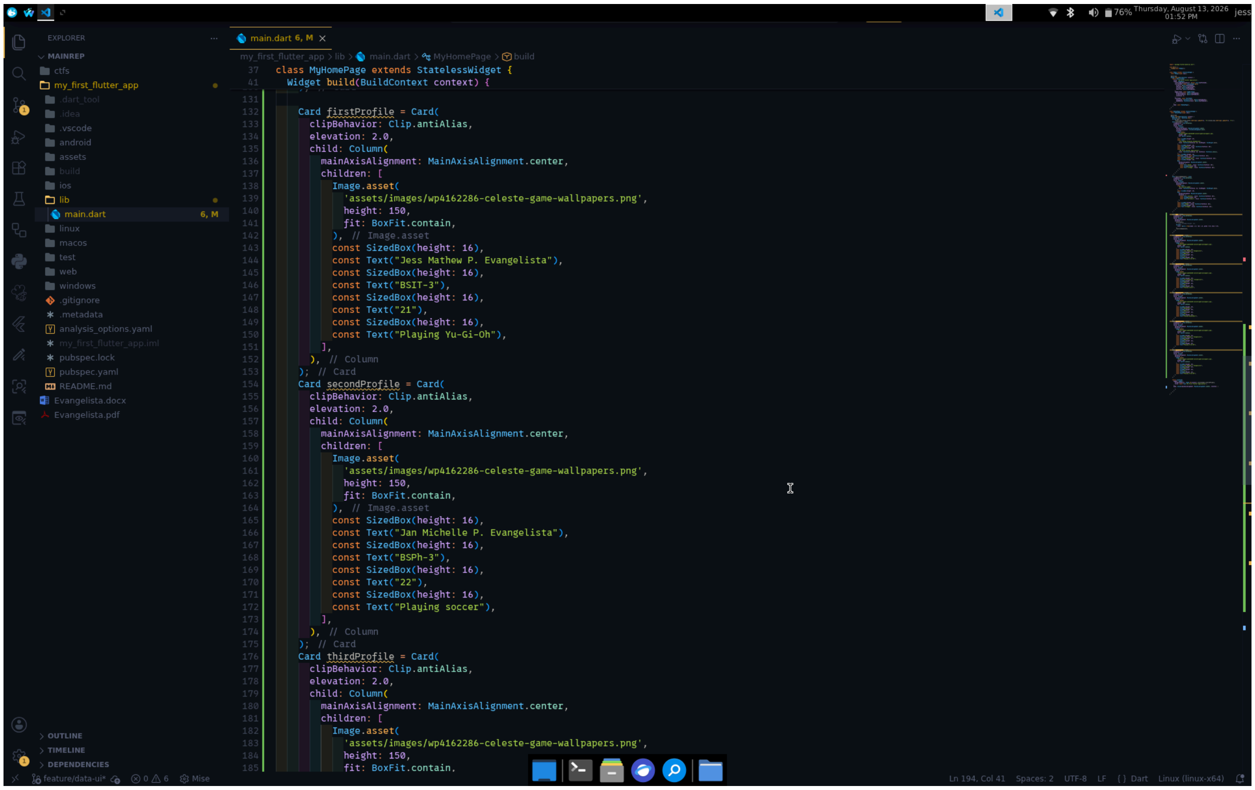
Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data



🏴 Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

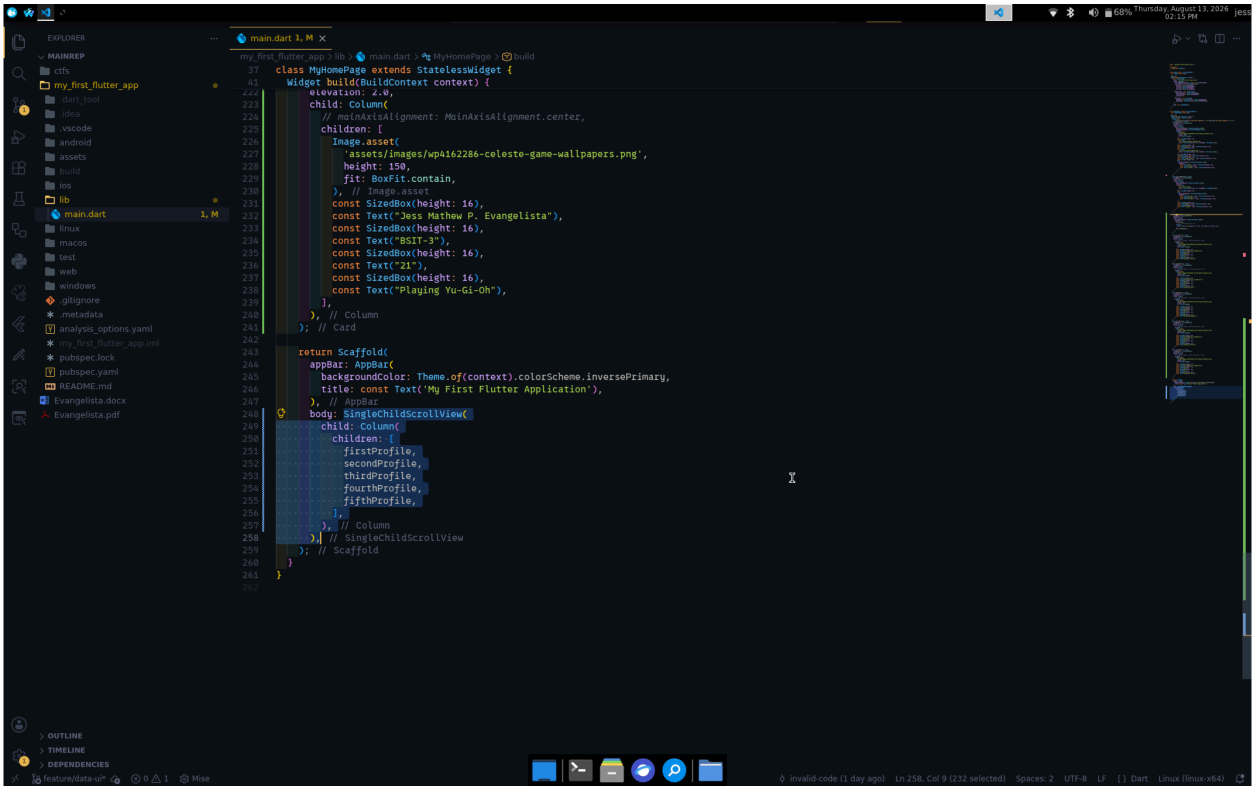
The user must be able to:

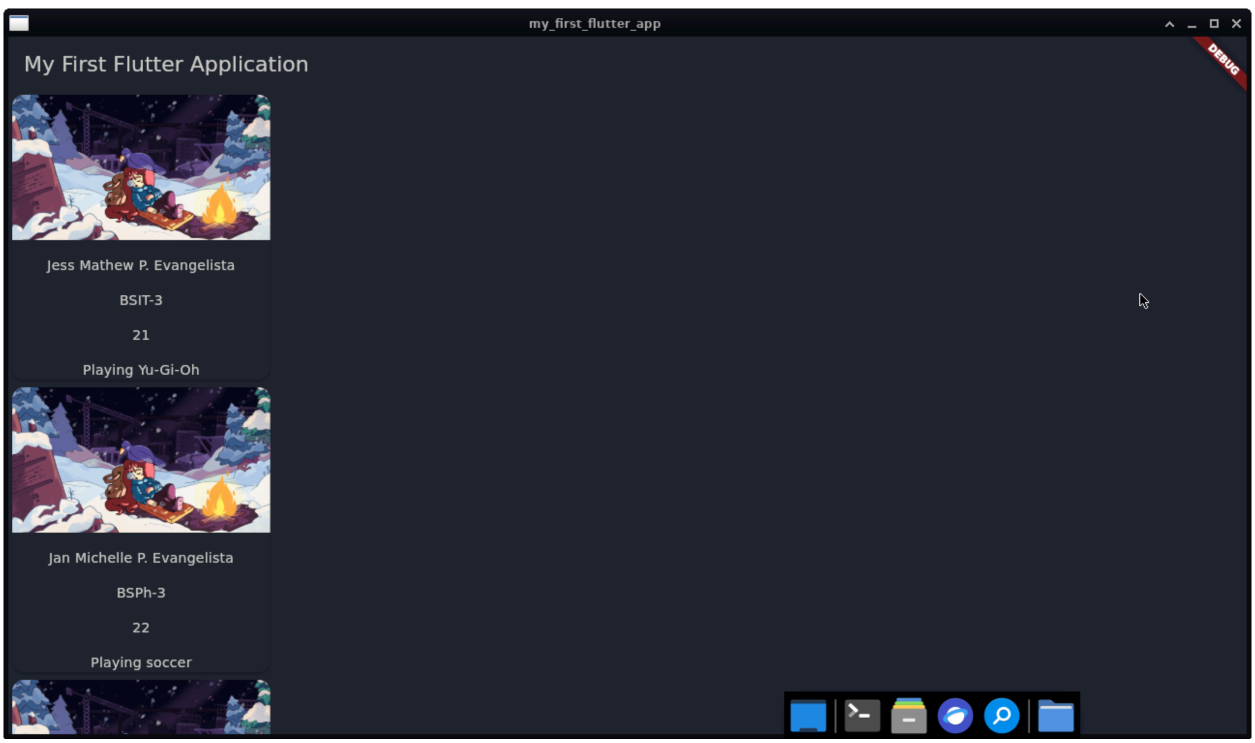
Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

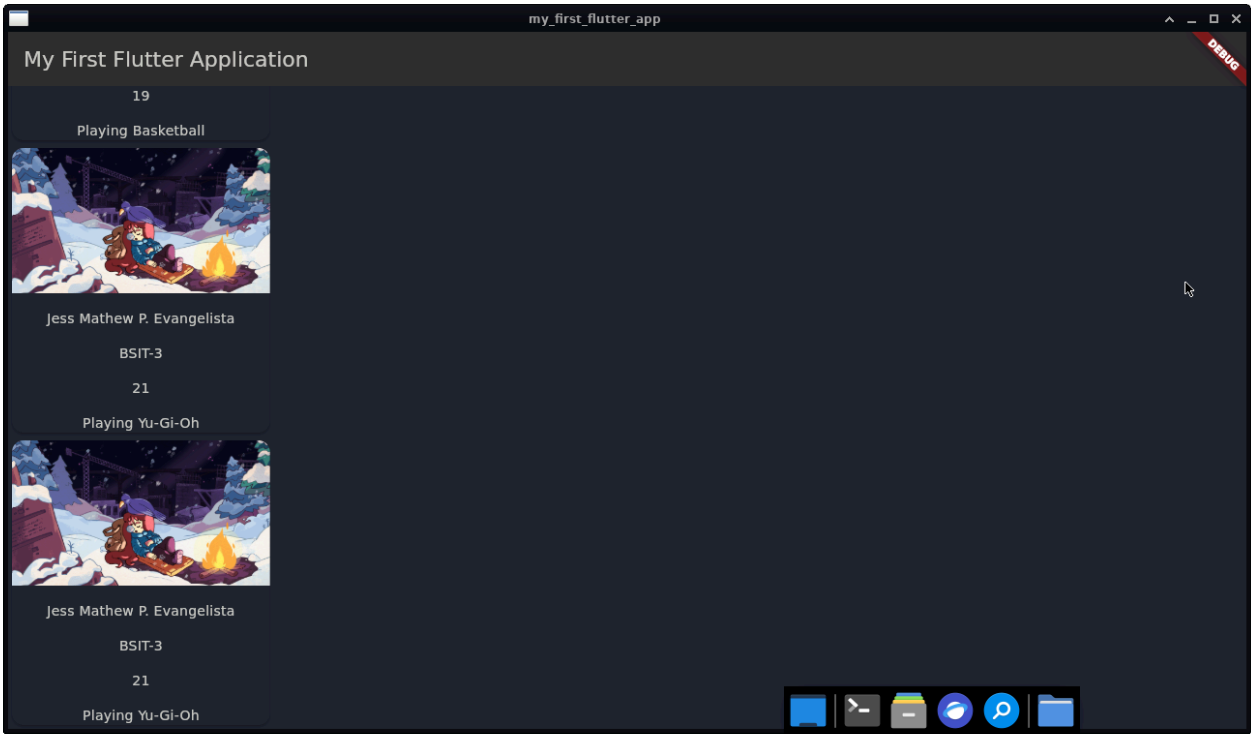
without changing the size of the screen.

Screenshot:

- The scrolling widget in your code
- Top portion of the application
- The rest of the portion of the application showing the later profiles


main.dart 1, M
EXPLORER
MAINREP
my_first_flutter_app
main.dart
class MyHomePage extends StatelessWidget {
Widget build(BuildContext context) {
child: Column(
// mainAxisAlignment: MainAxisAlignment.center,
children: [
Image.asset(
'assets/images/wp4162286-celeste-game-wallpapers.png',
height: 150,
fit: BoxFit.contain,
), // Image.asset
const SizedBox(height: 16),
const Text("Jess Mathew P. Evangelista"),
const SizedBox(height: 16),
const Text("BSIT-3"),
const SizedBox(height: 16),
const Text("21"),
const SizedBox(height: 16),
const Text("Playing Yu-Gi-Oh"),
],
), // Column
); // Card
return Scaffold(
appBar: AppBar(
backgroundColor: Theme.of(context).colorScheme.inversePrimary,
title: const Text('My First Flutter Application'),
), // AppBar
body: SingleChildScrollView(
child: Column(
children: [
firstProfile,
secondProfile,
thirdProfile,
fourthProfile,
fifthProfile,
],
), // Column
), // SingleChildScrollView
); // Scaffold
}
}
OUTLINE
TIMELINE
DEPENDENCIES


my_first_flutter_app
My First Flutter Application
DEBUG
Jess Mathew P. Evangelista
BSIT-3
21
Playing Yu-Gi-Oh
Jan Michelle P. Evangelista
BSPh-3
22
Playing soccer



🏴 Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A
Name: Maria
Course: BSIT

Hobby: missing

Profile B
Name: Juan
Course: missing
Hobby: Gaming

Profile C
Name: missing
Course: MMA
Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

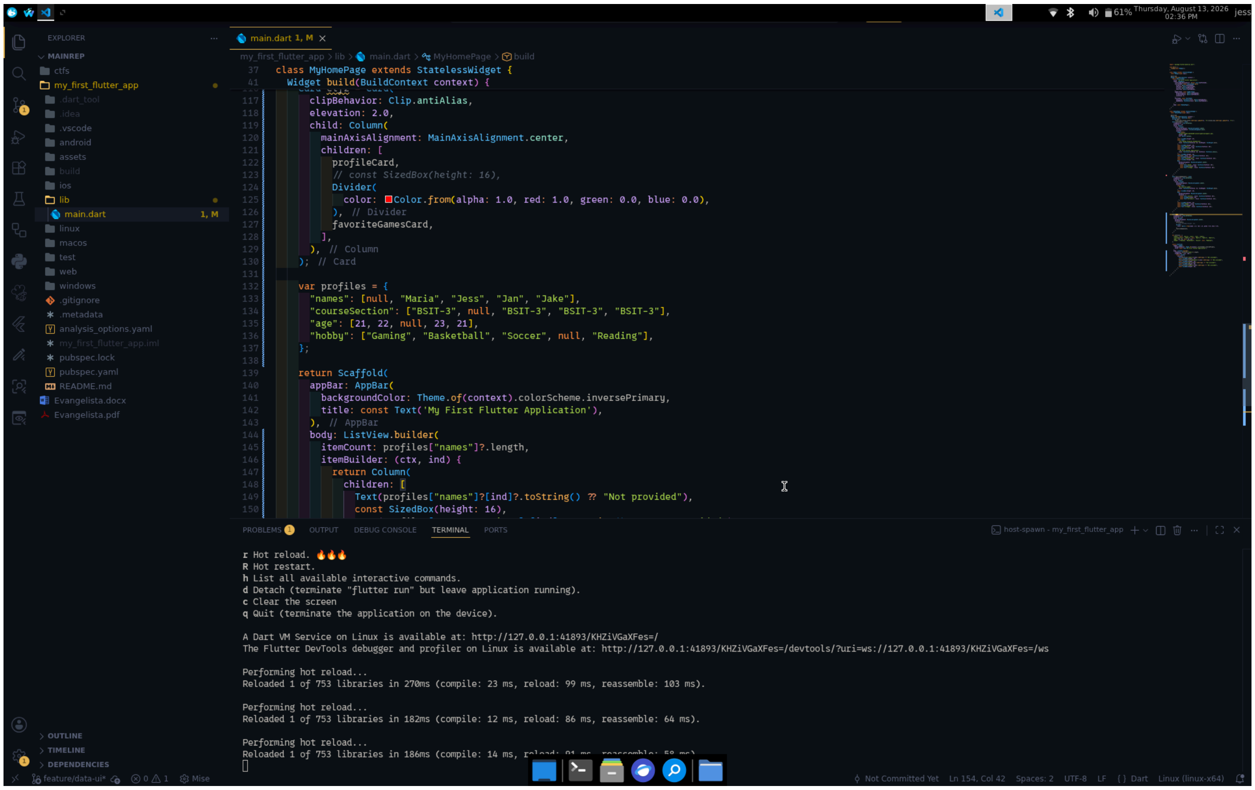
Hobby: Not provided
Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

- Incomplete profile data
- Application displaying fallback values
- Application still functioning correctly



🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit